



NEXORA

E-commerce Project Report

Architecture, features, security, deployment, verification, and a complete UI screenshot appendix.

PROJECT SNAPSHOT

A PostgreSQL-backed storefront with a Next.js interface, FastAPI services, customer commerce workflows, and an administrator area.

Prepared 07 October 2026

Prayukth & Suhal

1. Executive summary

NEXORA is a full-stack online storefront. Customers can discover products, manage a local or signed-in cart and wishlist, save addresses, and place stock-checked orders. Administrators manage products and orders through protected tools. The application is designed around PostgreSQL; SQLite is not used by the application test harness.

23	10	2	0
products returned by the local API during capture	primary routes captured	cart states captured	orders submitted during capture

Project scope

This report summarizes the current local application and its configured deployment approach. The included screenshots were captured from the running storefront at 1440 x 1100 in a signed-out browser context. A sample item was placed only in browser local storage to capture the populated cart and checkout; no order was submitted and no customer credentials were used.

2. Technology and architecture

Layer	Current implementation
Storefront	Next.js 16.4, React 19.3, TypeScript, Tailwind CSS 4, Framer Motion, Three.js, React Three Fiber and Drei.
API	Python FastAPI and Uvicorn; Pydantic request models; JWT authentication; bcrypt password hashes.
Database	PostgreSQL with SQLAlchemy 2 and Alembic migrations. Current local API response contained 23 products.
Supporting services	Redis for optional product caching and authentication rate limits; authenticated WebSockets for customer order updates.
Operations	Docker Compose configuration for PostgreSQL, Redis, backend API, and frontend.

At a high level, the browser talks to the FastAPI service. The service validates requests, applies role and stock rules, and reads/writes data in PostgreSQL through SQLAlchemy. Redis can provide caching and rate limiting when enabled.

3. Main capabilities

- **Catalog:** browse, search, filter, and sort by product attributes; product pages show price, availability, quantity, images, and related products.
- **Product presentation:** responsive product cards and detail screens, with category-aware 3D scenes and product-photo alternatives. Brand-specific iPhone/Samsung image selection is supported when those names appear in product data.
- **Customer commerce:** cart, wishlist, saved addresses, checkout, and order history. Checkout validates stock and stores a delivery-address snapshot with the order.
- **Administration:** role-protected product and order management plus summary sales statistics. Admin accounts are created or reset using the interactive bootstrap command; there is no fixed default admin password.
- **Live updates:** authenticated customers can receive order-created and order-status updates over WebSockets.
- **Shared UI:** navigation, search overlay, cart drawer, toast notifications, and the site-wide creator and copyright footer.

4. Security and deployment

Security controls implemented in the project include:

- JWT signing requires a configured secret of at least 32 bytes; production secrets are supplied through environment variables rather than source code.
- Passwords are hashed with bcrypt. Administrator creation and password reset use an interactive hidden password prompt; demo accounts are not created automatically.
- Role checks protect admin operations, and customers can access only their own commerce resources. Public storefront routes remain available without signing in.
- CORS is configurable as an explicit origin allowlist. Compose requires database and signing secrets, while the PostgreSQL host port is bound to loopback for local pgAdmin use.
- Demo product seeding is opt-in. Backend tests use a temporary PostgreSQL schema to avoid touching normal application tables.
- When Redis-backed authentication rate limiting is enabled, requests fail closed if the limiter is unavailable. Redis-backed product caching is optional.
- The WebSocket authentication token is carried in the URL query because browser WebSocket APIs cannot set authorization headers. Production must use WSS and prevent URL query strings from being logged; the current connection registry is process-local, so run one API worker unless cross-worker Pub/Sub is added.

Deployment checklist

- Generate unique, strong POSTGRES_PASSWORD and SECRET_KEY values; do not reuse development credentials.
- Set NEXT_PUBLIC_API to the public HTTPS API URL and CORS_ORIGINS to the exact HTTPS storefront origin before building.
- Provision PostgreSQL and Redis according to the Compose or managed-service design; apply Alembic migrations.
- Create the first administrator through the bootstrap CLI and verify the admin account and role before exposing admin access.
- Keep PostgreSQL off public interfaces, use HTTPS/WSS, protect environment files, and configure backups and operational monitoring.

5. Verification and current limits

Check	Observed result
Product API	GET /products?size=100 returned HTTP 200 and 23 catalog entries during screenshot capture.
Route rendering	Ten primary storefront routes returned HTTP 200. Guest requests for orders, account, and admin routes correctly redirected to the sign-in page.
Frontend type check and production build	Passed in the current working session after the latest image-mapping change.
Automated suites	Previously recorded project verification: 7 backend tests and 1 Playwright smoke test passed; dependency audits reported no known vulnerabilities at that validation point.

Known scope limits

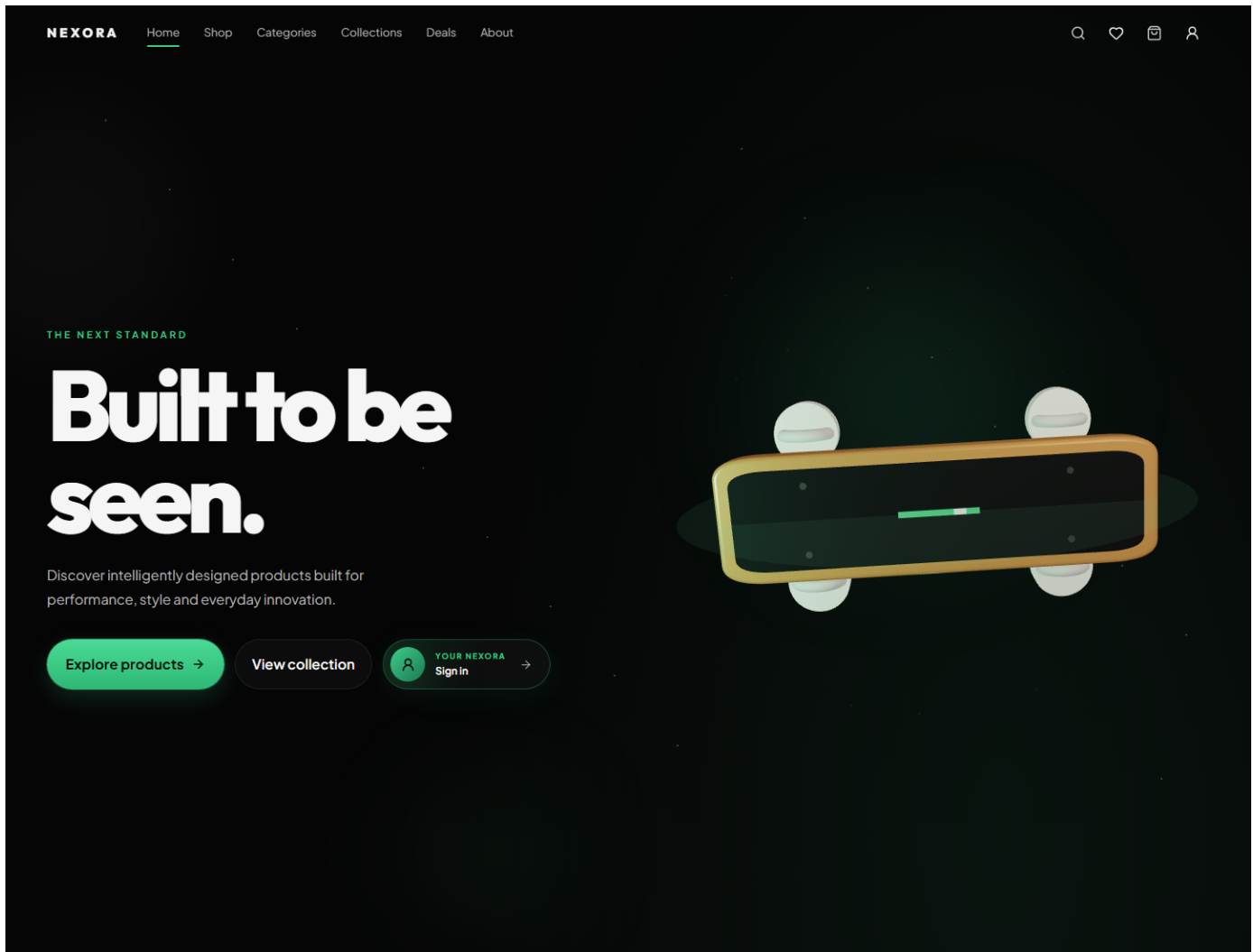
- Online payment processing is not integrated; checkout currently covers order data and delivery steps rather than payment-provider settlement.
- The report uses a signed-out browser session. The actual admin, customer-account, and order-history screens require valid roles and are not shown; their guest redirect is captured instead.
- The product data and screenshots reflect the local database state at capture time, not an external production environment.

Screenshot appendix

All screenshots below were captured from the running local storefront at 1440 x 1100. The two populated-cart views use one sample product in browser storage only. Product images are credited in the storefront footer; the iPhone and Samsung Commons photographs are licensed CC BY-SA 4.0.

Homepage and featured product area.

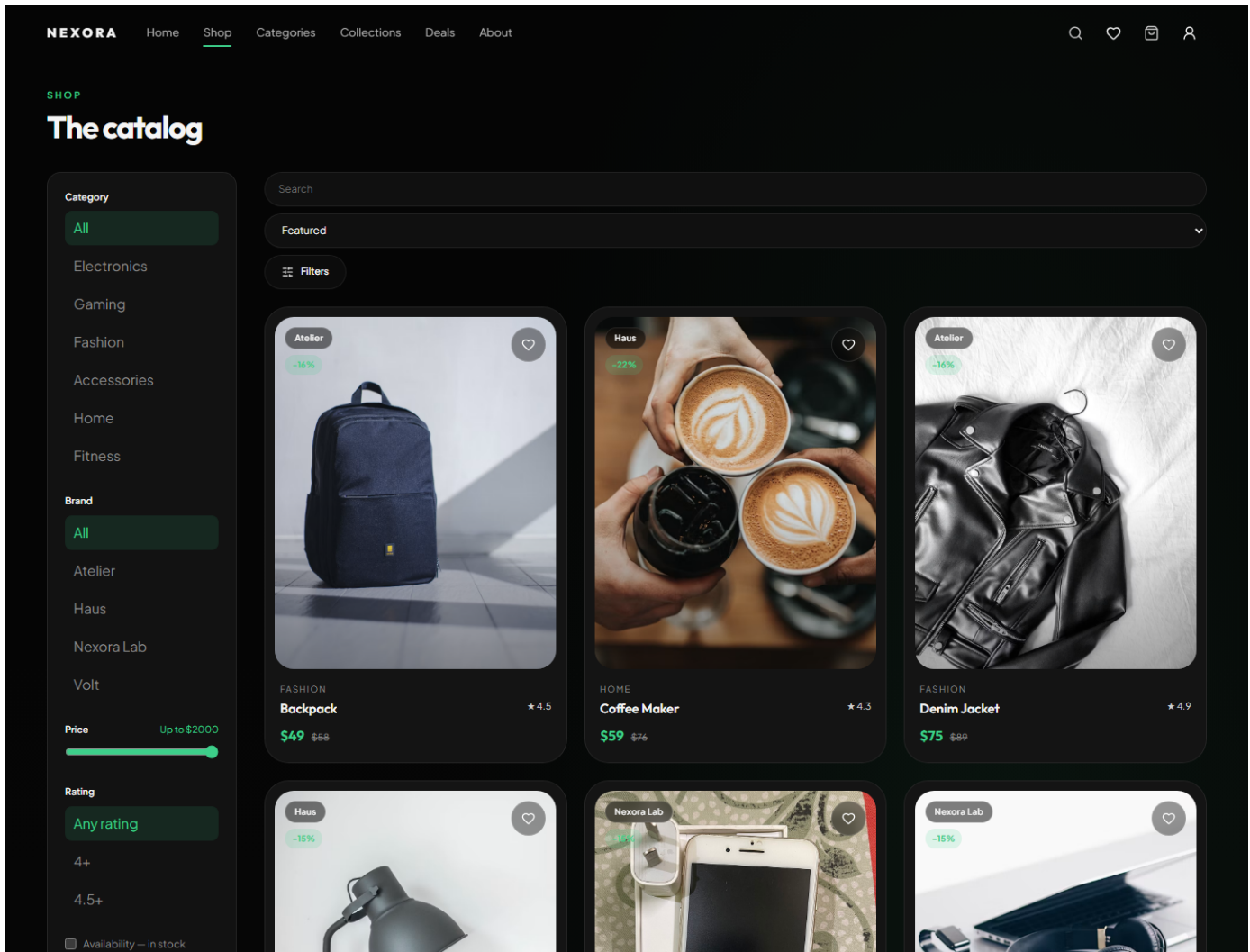
Route: / | Final page: http://127.0.0.1:3000/ | Capture: 1440 x 1100



Homepage and featured product area.

Catalog with product filters and visible product cards.

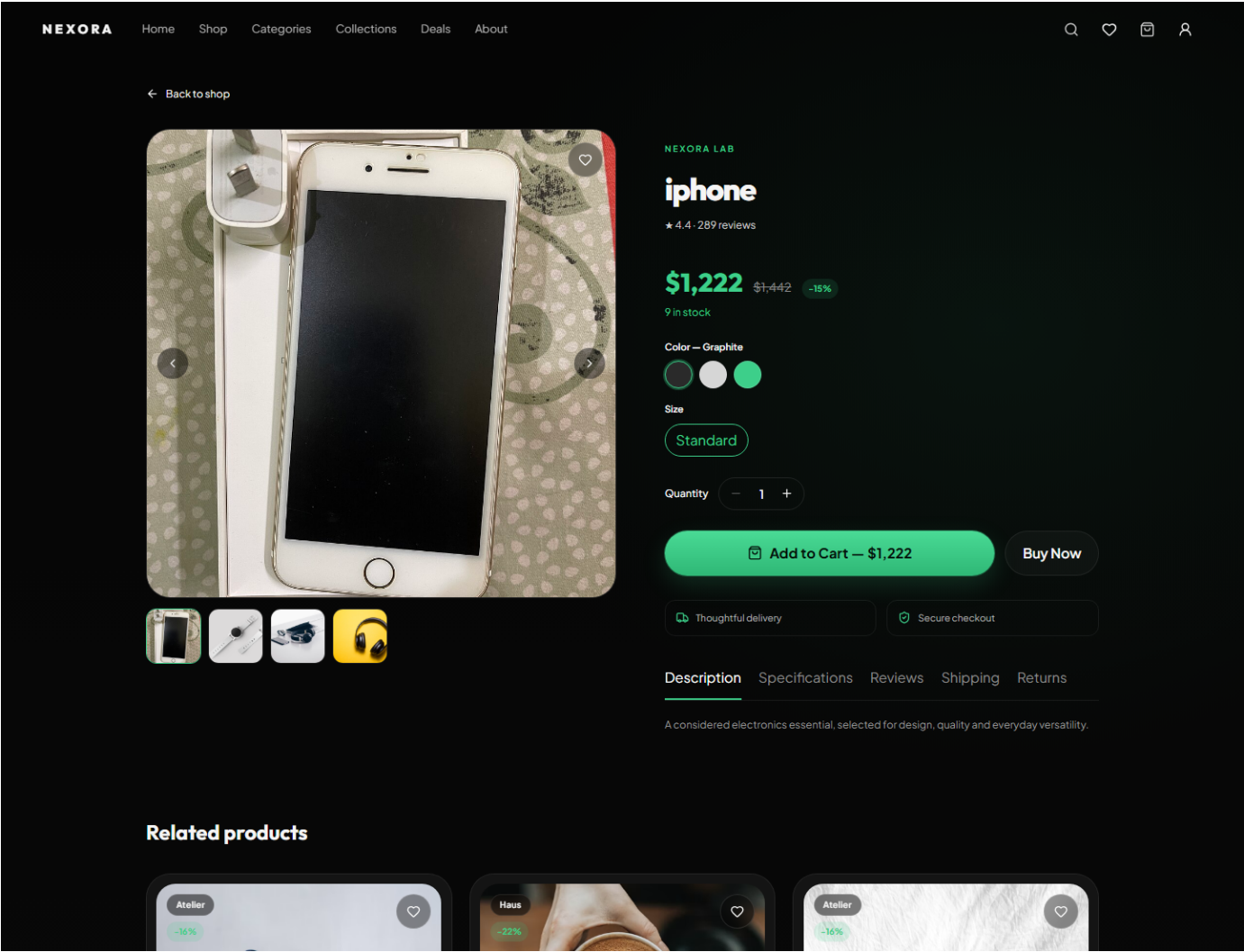
Route: /shop | Final page: http://127.0.0.1:3000/shop | Capture: 1440 x 1100



Catalog with product filters and visible product cards.

Product detail view for the current iPhone catalog entry.

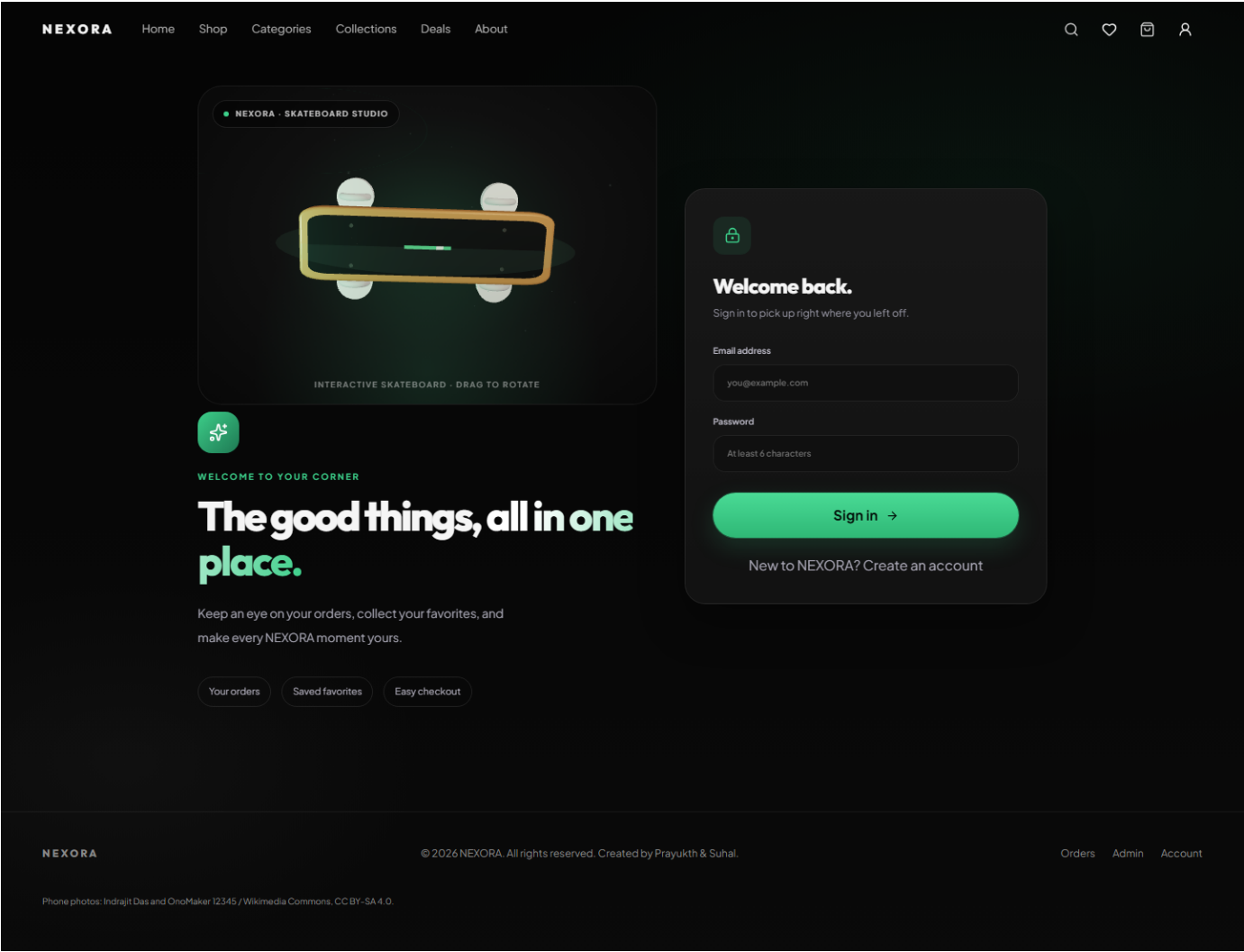
Route: /products/13 | Final page: http://127.0.0.1:3000/products/13 | Capture: 1440 x 1100



Product detail view for the current iPhone catalog entry.

Sign-in screen; guest state.

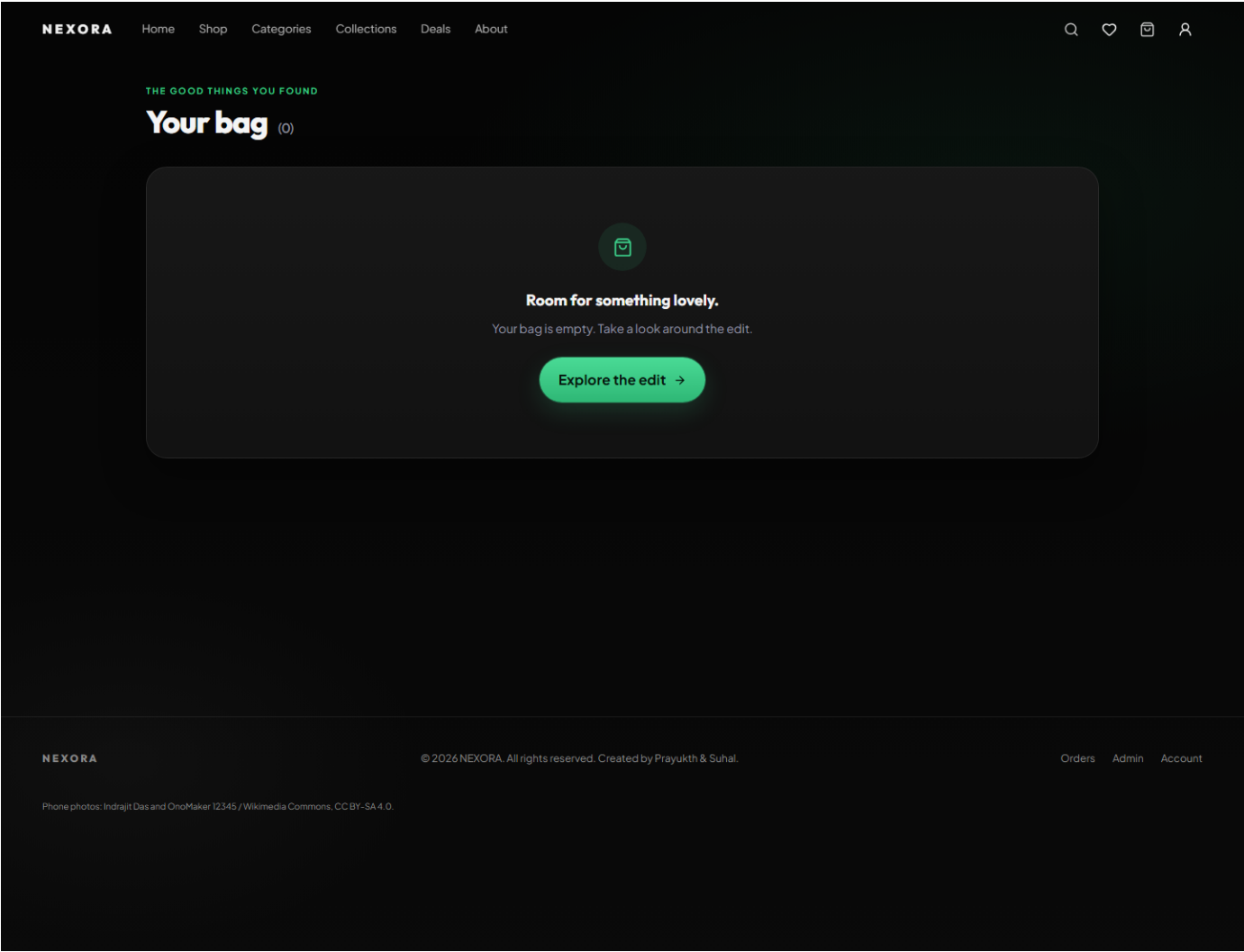
Route: /login | Final page: http://127.0.0.1:3000/login | Capture: 1440 x 1100



Sign-in screen; guest state.

Empty cart state.

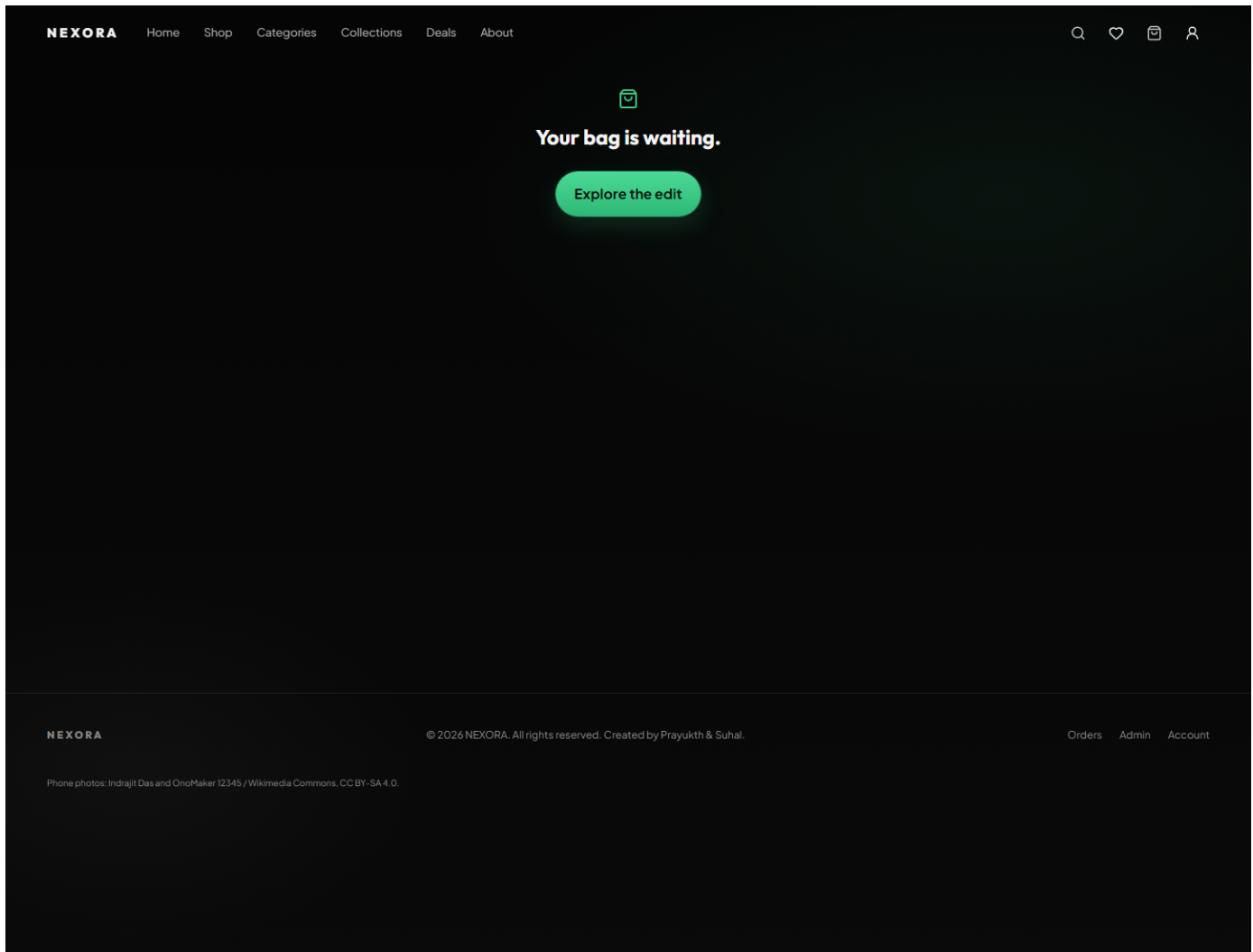
Route: /cart | Final page: http://127.0.0.1:3000/cart | Capture: 1440 x 1100



Empty cart state.

Checkout empty-cart state.

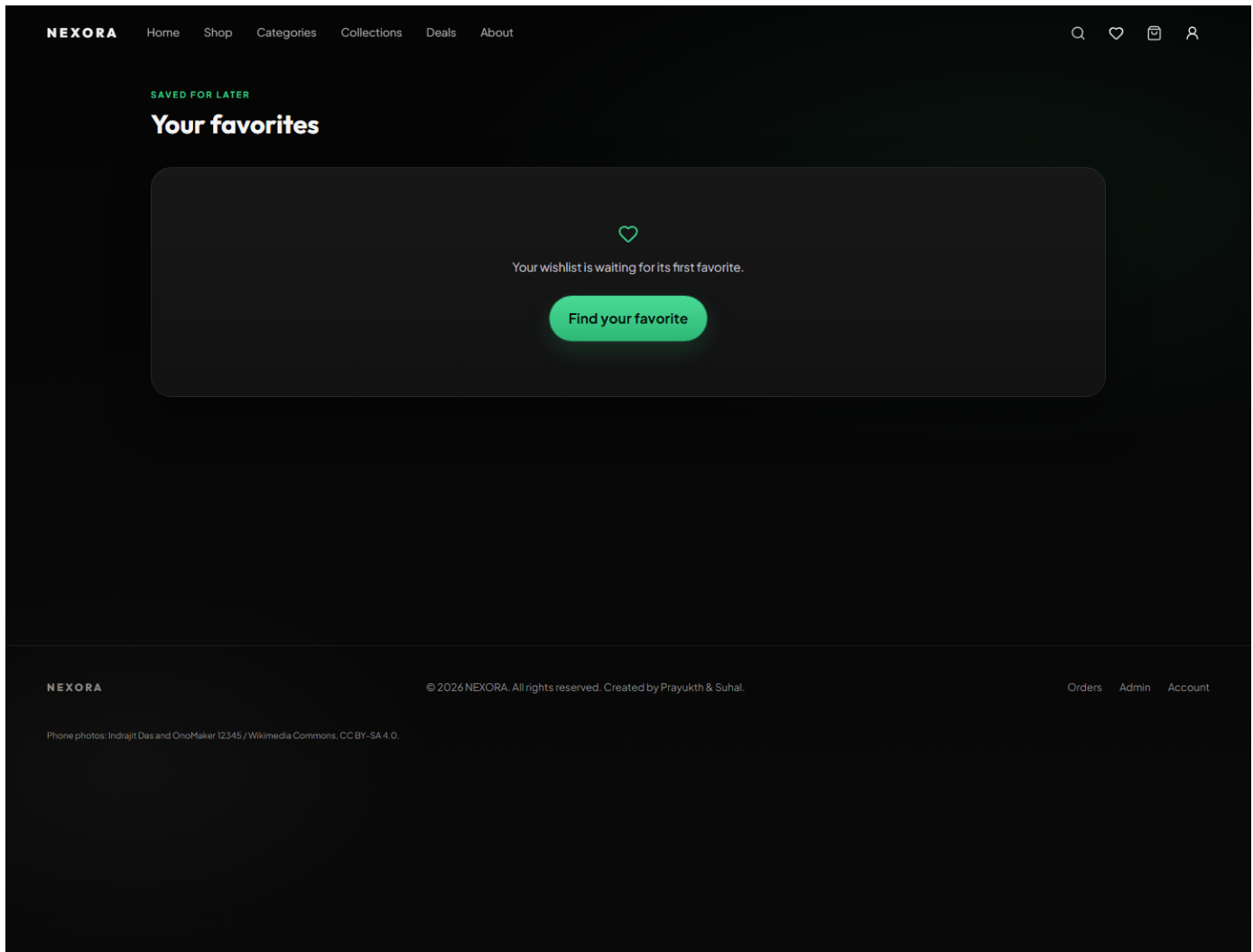
Route: /checkout | Final page: http://127.0.0.1:3000/checkout | Capture: 1440 x 1100



Checkout empty-cart state.

Empty wishlist state.

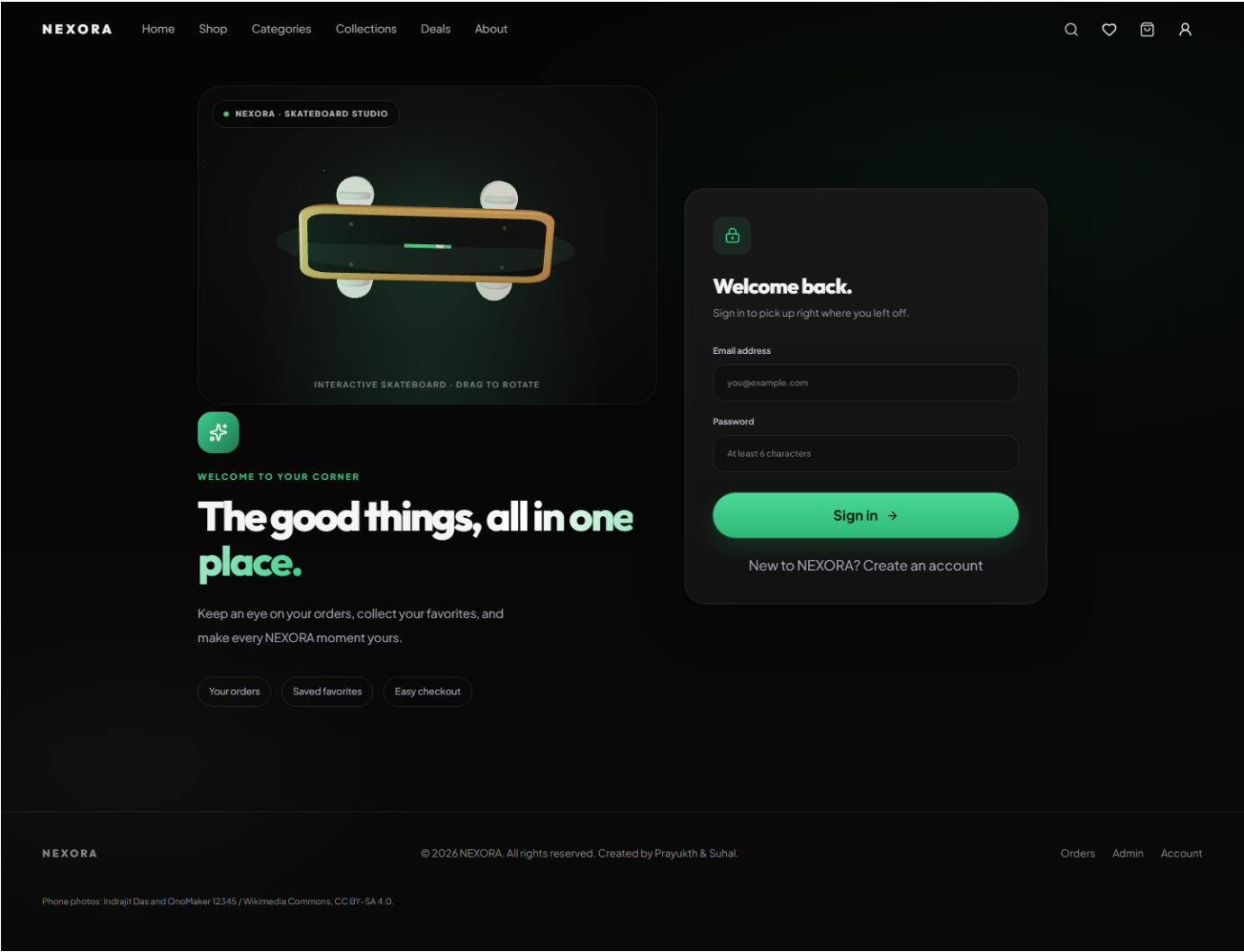
Route: /wishlist | Final page: http://127.0.0.1:3000/wishlist | Capture: 1440 x 1100



Empty wishlist state.

Orders route opened while signed out; guard redirects to sign-in.

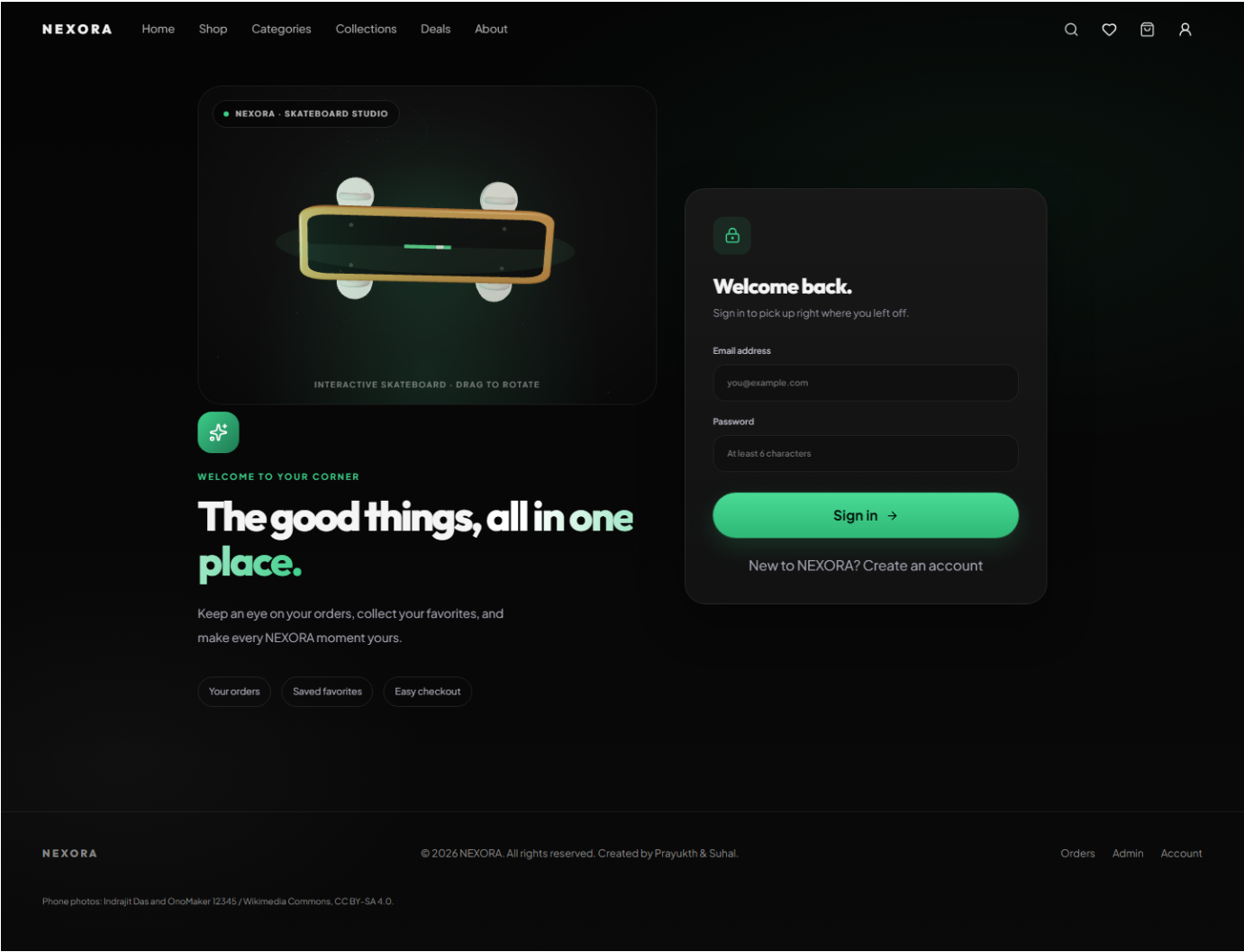
Route: /orders | Final page: http://127.0.0.1:3000/login | Capture: 1440 x 1100



Orders route opened while signed out; guard redirects to sign-in.

Account route opened while signed out; guard redirects to sign-in.

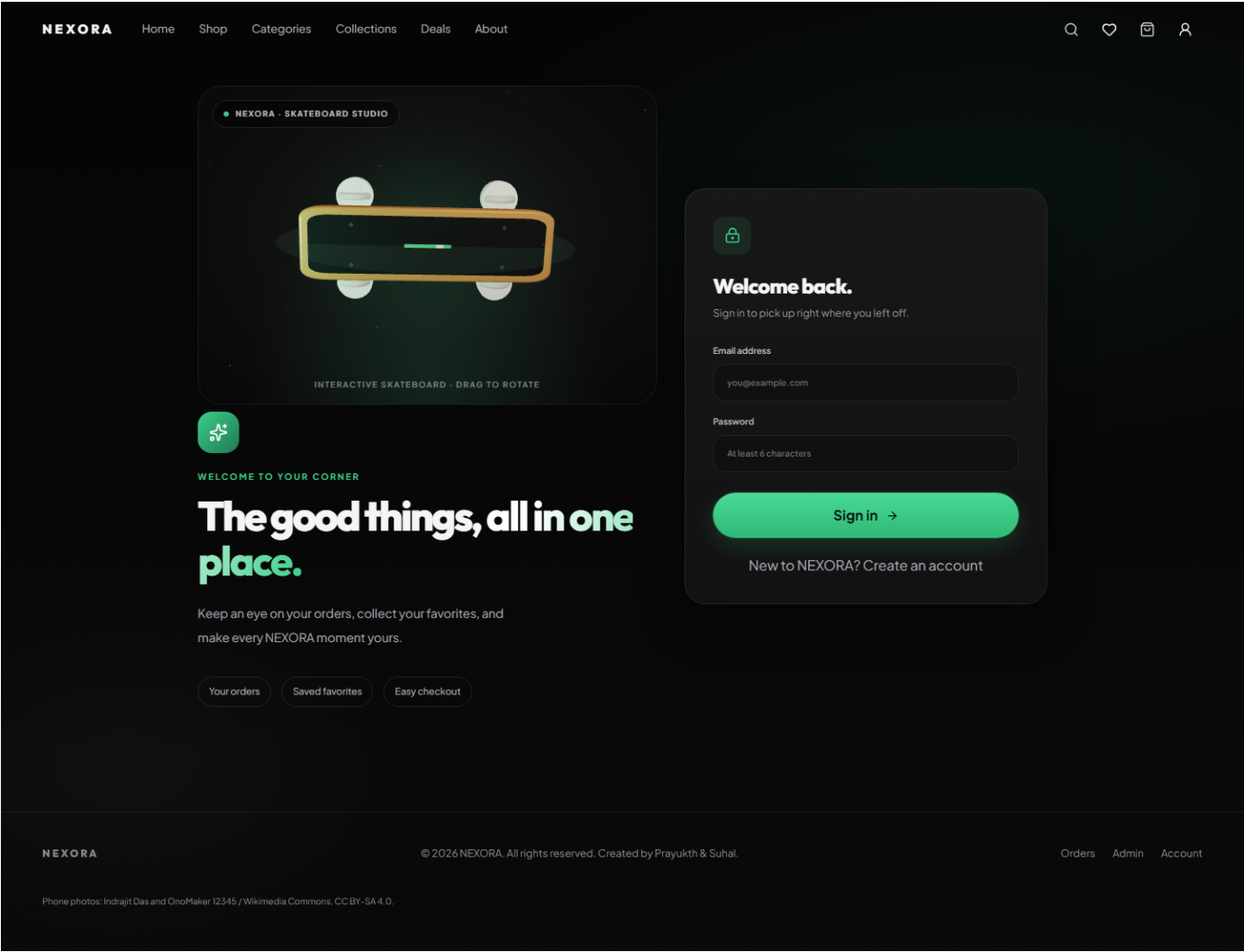
Route: /account | Final page: http://127.0.0.1:3000/login | Capture: 1440 x 1100



Account route opened while signed out; guard redirects to sign-in.

Admin route opened while signed out; guard redirects to sign-in.

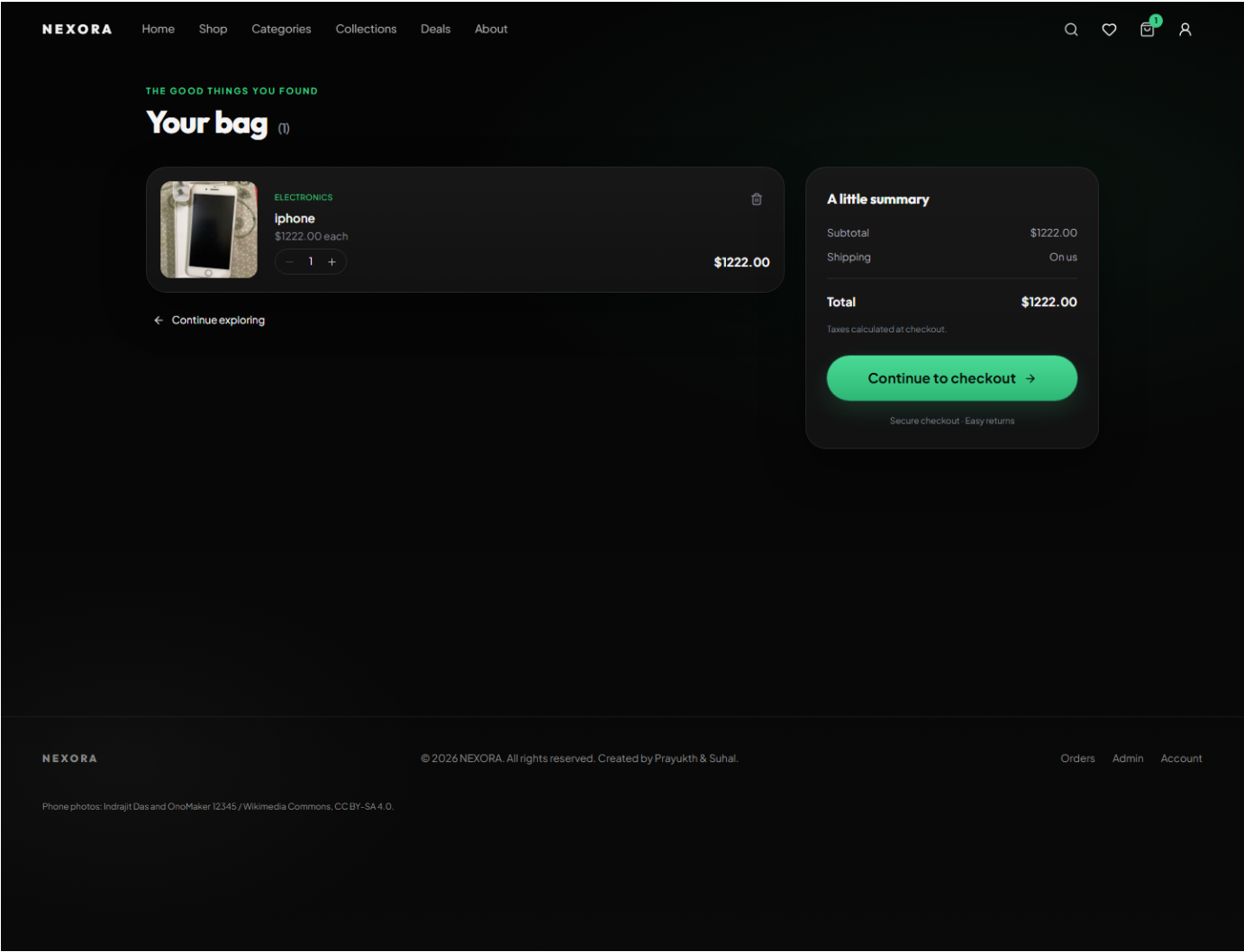
Route: /admin | Final page: http://127.0.0.1:3000/login | Capture: 1440 x 1100



Admin route opened while signed out; guard redirects to sign-in.

Cart containing one sample product; state stored locally in the browser.

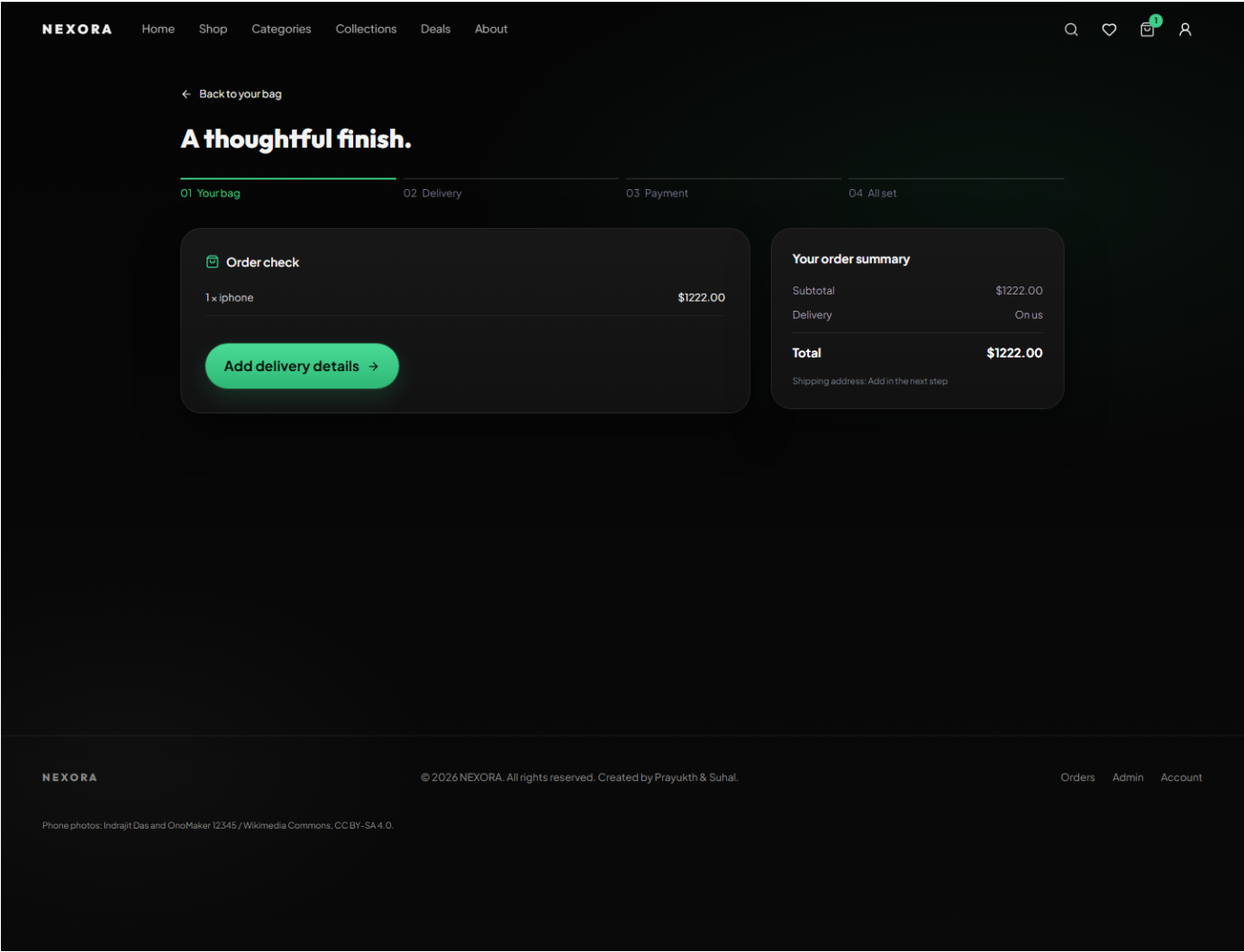
Route: /cart | Final page: http://127.0.0.1:3000/cart | Capture: 1440 x 1100



Cart containing one sample product; state stored locally in the browser.

Checkout order summary with one sample product; no order was submitted.

Route: /checkout | Final page: http://127.0.0.1:3000/checkout | Capture: 1440 x 1100



Checkout order summary with one sample product; no order was submitted.